

# AI 40 Interview Questions and Answers



## Q1. What is the difference between AI, ML, DL and GenAI?

AI makes machines intelligent. ML is a subset of AI that learns from data. DL uses deep neural networks, while GenAI creates new content like text, images, and code.



## Q2. What are the types of Machine Learning?

Supervised learning uses labeled data. Unsupervised learning finds hidden patterns. Reinforcement learning learns through rewards and penalties.



## Q3. What is overfitting?

Overfitting happens when a model memorizes training data. It performs well on training but poorly on new data. Regularization and more data help reduce it.



## Q4. Explain Bias-Variance Tradeoff.

High bias causes underfitting, while high variance causes overfitting. A good model balances both. Cross-validation helps choose the right complexity.



## Q5. Classification vs Regression

Classification predicts categories like spam or not spam. Regression predicts continuous values like house prices. Both are supervised learning tasks.



## Q6. How do you handle imbalanced datasets?

Use oversampling, undersampling, or SMOTE. Choose suitable evaluation metrics like F1-score. Class weights also improve performance.



## Q7. Evaluation metrics

Accuracy, Precision, Recall, and F1-score are used for classification. MAE, MSE, and RMSE are common regression metrics. Choose metrics based on the problem.



## Q8. What is Cross Validation?

Cross-validation splits data into multiple folds. The model trains and validates on different folds. It gives a reliable estimate of performance.



## Q9. Working of Gradient Descent

Gradient descent updates model weights to reduce error. It moves in the direction of the negative gradient. Learning rate controls update size.



## Q10. Batch vs Mini-batch vs SGD

Batch uses all data at once. SGD updates after every sample. Mini-batch uses small groups and is the most common approach.



## Q11. What is a Neural Network?

A neural network consists of input, hidden, and output layers. It learns by adjusting weights. It is inspired by the human brain.



## Q12. CNN vs RNN vs Transformer

CNN is best for images. RNN is designed for sequential data. Transformers process sequences faster using attention.



## Q13. Backpropagation

Backpropagation calculates prediction error. It updates weights using gradients. This helps the model learn.



## Q14. Activation Functions

Activation functions introduce non-linearity. ReLU is widely used because it is simple and efficient. Sigmoid and Tanh are used in specific cases.



## Q15. Vanishing and Exploding Gradients

Very small gradients slow learning, while very large gradients make training unstable. ReLU, BatchNorm, and gradient clipping help solve this.



## Q16. Dropout and Batch Normalization

Dropout randomly disables neurons to reduce overfitting. Batch Normalization stabilizes learning. Together they improve performance.



## Q17. Learning Rate

Learning rate controls how much weights change. Too high causes instability. Too low makes training slow.



## Q18. Transfer Learning

Transfer learning reuses a pre-trained model. It saves training time. It works well with limited data.



## Q19. Fine-tuning in LLMs

Fine-tuning trains a pre-trained LLM on task-specific data. It improves performance for a particular application. LoRA is a popular efficient method.



## Q20. Tokenization vs Embedding

Tokenization splits text into tokens. Embeddings convert tokens into numerical vectors. Models understand text using embeddings.



## Q21. What are LLMs?

LLMs are AI models trained on huge text datasets. They understand and generate human-like language. Examples include GPT and Llama.



## Q22. How do LLMs work?

LLMs predict the next token using transformer architecture. They learn language patterns from massive datasets. Training uses self-attention.



## Q23. Zero-shot, One-shot and Few-shot

Zero-shot uses only instructions. One-shot provides one example. Few-shot provides multiple examples to improve results.



## Q24. Prompt Engineering

Prompt engineering is writing effective instructions for AI models. Better prompts produce better outputs. It improves response quality.



## Q25. RAG vs Fine-tuning

RAG retrieves external knowledge during inference. Fine-tuning changes model weights. RAG is cheaper for frequently changing data.



## Q26. Hallucinations

Hallucinations are incorrect or invented answers. Better prompts, RAG, and verification reduce them. High-quality data also helps.



## Q27. RAG Pipeline

Load documents, create embeddings, store them in a vector database, retrieve relevant chunks, and send them with the prompt to the LLM.



## Q28. Embeddings

Embeddings are dense numerical vectors representing meaning. Similar texts have similar vectors. They are widely used in search and RAG.



## Q29. Context Window

The context window is the amount of text an LLM can process at once. Larger windows allow longer conversations and documents.



### Q30. LLMs worked with

Common LLMs include GPT, Llama, Mistral, Qwen, Gemma, and Claude. Choose based on cost, accuracy, and deployment needs.



### Q31. LangChain

LangChain helps build LLM applications. It connects prompts, models, tools, memory, and retrievers. It simplifies AI workflows.



### Q32. LangGraph

LangGraph builds stateful AI agents using graphs. It supports loops and branching. It is useful for multi-agent systems.



### Q33. API vs SDK

An API defines how applications communicate. An SDK provides tools, libraries, and APIs for development. SDKs make integration easier.



### Q34. Deploy an LLM application

Package the app with FastAPI, Docker, and deploy to cloud platforms. Monitor logs and scale based on traffic. Secure API keys.



### Q35. FastAPI

FastAPI is a modern Python web framework. It builds fast REST APIs. It is widely used in AI backends.



### Q36. LLMs with databases/APIs

LLMs fetch data from databases or APIs through backend code. The retrieved information is added to the prompt. This enables dynamic responses.



### Q37. Docker

Docker packages applications with dependencies. It ensures consistent execution. It simplifies deployment.



## Q38. Monitoring AI apps

Monitor latency, token usage, errors, and user feedback. Use logging and dashboards. Track model quality continuously.



## Q39. CI/CD in ML

CI/CD automates testing, training, and deployment. It reduces manual work. It ensures reliable model updates.



## Q40. Vector Databases

Vector databases store embeddings for semantic search. Examples include Pinecone, Chroma, FAISS, and Weaviate.

# Coding Questions



## Reverse String

```
s='hello'  
print(s[::-1])
```

`[::-1]` reverses the string. Time Complexity:  $O(n)$ .



## Palindrome

```
s='madam'  
print(s==s[::-1])
```

Compare the string with its reverse. Equal means palindrome.



## Second Largest

```
a=[5,2,9,7]  
print(sorted(set(a))[-2])
```

Remove duplicates, sort the list, and pick the second largest.



## Remove Duplicates

```
a=[1,2,2,3]
print(list(dict.fromkeys(a)))
```

Keeps only unique values while preserving order.



## Merge Sorted Lists

```
a=[1,3];b=[2,4]
print(sorted(a+b))
```

Merge both lists and sort them.



## Recursive Factorial

```
def fact(n):
    return 1 if n<=1 else n*fact(n-1)
print(fact(5))
```

Uses recursion until the base case is reached.



## Two Sum

```
nums=[2,7,11,15];target=9
seen={}
for x in nums:
    if target-x in seen:
        print(target-x,x);break
    seen[x]=1
```

Dictionary gives O(n) solution by storing visited numbers.



## LRU Cache

```
from functools import lru_cache
@lru_cache(None)
def fib(n):
    return n if n<2 else fib(n-1)+fib(n-2)
```

lru\_cache stores previous results to avoid recomputation.



## Serialize Tree

```
# Preorder traversal with '#' for null nodes.
```

Store node values and null markers so the tree can be rebuilt.



## FastAPI API

```
from fastapi import FastAPI
app=FastAPI()
```

```
@app.get('/')
def home():
    return {'message': 'Hello'}
```

Create the app, define a GET endpoint, and run using uvicorn.